

Guided OS Project: Multi-Level Feedback Queue Scheduler on xv6-RISC-V

Group members:

1. Abdullah Irfan 29266
2. Muhammad Fahad 29264

Note: the following repository was cloned as the basis for the project:

<https://github.com/mit-pdos/xv6-riscv.git>

INDEX/APPENDIX

Part 1: Queue Structure:

Part 2: Time Quanta

Part 3: Process Lifecycle

Part 4: Data Structures:

Part 5: Key Functions

Part 6: Scheduling Algorithm:

Part 7: Testing Plan

Part 8 Expected Behavior

Part 9: Syscal output screenshot

Week 1: Setup and Design → Design Document

MLFQ Configuration

Part 1: Queue Structure:

- Number of Queues: 4 (Queue 0 to Queue 3)
- Priority: Queue 0 is highest priority, Queue 3 is lowest
- Implementation strategy:
 1. lists using next_proc pointer in struct proc
 2. No array shifting required
 3. O(1) enqueue and dequeue operations
 4. Each queue maintains separate head and tail pointers
- Data Structure:

```
// Queue structure for MLFQ
struct mlfq_queue {
    struct proc *head;           // First process in queue
    struct proc *tail;          // Last process in queue
};

// The 4 priority queues
static struct mlfq_queue mlfq[MLFQ_LEVELS];
```

Part 2: Time Quanta

Level	Quantum (ticks)	Formula	Purpose
0	4	4×2^0	Highest priority, shortest quantum for responsiveness
1	8	4×2^1	Medium-high priority
2	16	4×2^2	Medium-low priority
3	32	4×2^3	Lowest priority, longest quantum for throughput

- ✓ Short quanta for high priority → Better responsiveness for interactive processes
- ✓ Long quanta for low priority → Less context switching overhead for CPU-bound processes
- ✓ Exponential growth → Balances fairness with efficiency
- ✓ Minimum 4 ticks → Avoids excessive context switching (1-2 ticks too short for practical systems)

Q Why not start at 1 tick?

- 1 tick quantum causes excessive context switching overhead
- Process barely gets to execute before being preempted
- 4 ticks provides reasonable work unit while maintaining responsiveness

```
// Time quantum for each level
static int time_quanta[MLFQ_LEVELS] = {4, 8, 16, 32};
```

Part 3: Process Lifecycle

1. New Process Creation

- Initial state: Level = 0 (highest priority), ticks_used = 0, quantum = 4
- Action: Added to tail of queue 0
- Reasoning: New processes may be interactive; short tasks finish faster; CPU-bound tasks quickly demoted.

2. Running Process

- Each timer tick → ticks_used++
- When process yields or is preempted:

Demotion Rule

```
If ticks_used ≥ quantum:
    Move to next lower level (max = 3)
Else:
    Requeue at same level
```

Afterwards: reset ticks_used = 0, update quantum, enqueue at tail.

3. I/O-Blocked Process

When sleeping:

- State → SLEEPING
- Removed from queue
- Keep same priority
- Reset ticks_used = 0

When waking:

- Re-queued **at the same level**
- Reason: I/O-bound processes voluntarily yield → considered “good behavior,” should not be demoted.

4. Demotion Policy (CPU-bound Detection)

A process is demoted if it **uses its entire quantum**.

```
if (ticks_used >= quantum) {
    if (queue_level < 3)
        queue_level++
    quantum = get_quantum(queue_level)
    ticks_used = 0
    enqueue(process)
}
```

Example (CPU-bound)

- Start: Q0 → use 4 ticks → Q1
- Q1 → use 8 ticks → Q2
- Q2 → use 16 ticks → Q3
- Q3 → always stays in Q3 (long-term CPU-bound)

5. Starvation Prevention (Priority Boosting)

Problem: Low-priority processes may never run.

Solution: Global priority boost every 200 ticks

- Move all RUNNABLE/RUNNING processes to queue 0
- Reset: ticks_used = 0, quantum = 4
- Reset boost counter

Rationale

- Stops starvation
- Keeps system responsive
- CPU-bound tasks get temporary priority but eventually fall back down

Part 4: Data Structures:

Modified struct proc (kernel/proc.h)

```
// MLFQ Scheduler Fields
int queue_level;           // Current queue level (0-3, 0 is highest priority)
int ticks_used;            // Ticks consumed in current queue
int quantum;               // Time quantum for current queue level
struct proc *next_proc;    // Next process in queue (linked list)
```

- a) queue_level determines which queue process belongs to
- b) ticks_used compared against quantum to trigger demotion
- c) quantum stored per-process for fast lookup (avoids repeated calculation)
- d) next_proc implements efficient FIFO queues

Global Data Structures (kernel/proc.c)

```

#define MLFQ_LEVELS 4
#define BOOST_INTERVAL 200

// Queue structure for MLFQ
struct mlfq_queue {
    struct proc *head;    // First process in queue
    struct proc *tail;    // Last process in queue
};

// The 4 priority queues
static struct mlfq_queue mlfq[MLFQ_LEVELS];

// Global boost counter
//static uint boost_counter = 0;

// Time quantum for each level
static int time_quantum[MLFQ_LEVELS] = {4, 8, 16, 32};

```

Part 5: Key Functions

Queue Management Functions

1. `int get_quantum(int level)` → Returns time quantum for given priority level.
2. `void mlfq_enqueue(struct proc *p, int level)`; → Adds process to end of queue at given level (FIFO order).
3. `struct proc* mlfq_dequeue(int level)`; → Removes and returns first process from queue.
4. `void mlfq_remove(struct proc *p)`; → Removes specific process from its current queue.
5. `struct proc* mlfq_next(void)`; → Returns next runnable process from highest priority queue.

Scheduling Functions

1. `void mlfq_demote(struct proc *p){}`; → Demotes process to next lower priority level.
2. `void mlfq_boost(void)` → Boosts all processes to highest priority (starvation prevention).
3. `scheduler()` → Main scheduler loop - selects and runs processes.
4. `yield()` → Process gives up CPU - checks for demotion.

System Call

`int getprocinfo(struct procinfo *info)`; → Returns MLFQ state of current process for testing and debugging.

Part 6: Scheduling Algorithm (logics) :

Scheduler Loop (kernel/proc.c)

```

function scheduler():
    loop forever:
        enable_interrupts()

        p = mlfq_next() // Get highest priority process

        if p != NULL:
            acquire(p->lock)
            p->state = RUNNING
            context_switch(scheduler_context, p->context)
            // Process runs until it yields
            release(p->lock)
        else:
            // No runnable processes
            wait_for_interrupt()

```

Timer Interrupt Handler (kernel/trap.c)

```

function handle_timer_interrupt():
    p = current_process

    // Priority boosting check
    boost_counter++
    if boost_counter >= BOOST_INTERVAL:
        mlfq_boost()
        boost_counter = 0

    // Update current process accounting
    p->ticks_used++
    |
    // Check for demotion
    if p->ticks_used >= p->quantum:
        if p->queue_level < 3:
            p->queue_level++
            p->quantum = get_quantum(p->queue_level)
        p->ticks_used = 0
    yield() // Give up CPU

```

Process Yield (kernel/proc.c)

```

function yield():
    acquire(p->lock)

    // Check if used full quantum (demotion)
    if p->ticks_used >= p->quantum:
        mlfq_demote(p)

    // Make process runnable and enqueue
    p->state = RUNNABLE
    mlfq_enqueue(p, p->queue_level)

    // Switch back to scheduler
    context_switch(p->context, scheduler_context)

    release(p->lock)

```

Sleep (I/O) Handling (kernel/proc.c)

```
function sleep(chan, lock):
    acquire(p->lock)

    p->state = SLEEPING
    p->chan = chan

    // Reset ticks but KEEP priority
    p->ticks_used = 0

    // Switch to scheduler
    context_switch(p->context, scheduler_context)
    release(p->lock)

function wakeup(chan):
    for each process p:
        if p->state == SLEEPING and p->chan == chan:
            p->state = RUNNABLE
            // Re-enqueue at CURRENT level (reward I/O behavior)
            mlfq_enqueue(p, p->queue_level)
```

Part 7: Testing Plan

Test 1: Single CPU-Bound Process

Goal: Verify demotion.

Expected:

Starts at queue 0, drops to 1 → 2 → 3
Boosted to queue 0 every 200 ticks
Repeats cycle

Validate:

Use `getprocinfo()` to track `queue_level` + `ticks_used`; confirm demotion at correct quantum boundaries.

Test 2: Single I/O-Bound Process

Goal: Ensure I/O-bound processes stay high-priority.

Expected:

Frequent sleep keeps it in queue 0 or 1
Finishes quickly

Validate:

Check queue levels stay low; compare runtime vs CPU-bound process.

Test 3: Mixed Workload

Goal: Check fairness between CPU-bound & I/O-bound.

Setup: One busy-loop process + one frequent-sleep process.

Expected:

CPU-bound demotes to queue 3
I/O-bound stays near queue 0
I/O-bound finishes first

Validate:

Monitor via `getprocinfo()`, record completion times.

Test 4: Starvation Prevention

Goal: Test 4: Starvation Prevention

Setup: Several CPU-bound processes all demoting to queue 3.

Expected:

All processes continue to run
Every 200 ticks → all reset to queue 0
No starvation

Validate:

Track priority levels; verify periodic boosts and eventual completion.

Part 8 Expected Behavior:

CPU-Bound Process Timeline			
Time (ticks)	Queue	Ticks Used	Action
0	0	0	Process starts
4	1	0	Demoted (used full quantum)
12	2	0	Demoted (used full quantum)
28	3	0	Demoted (used full quantum)
60	3	0	Stays at lowest (used full quantum)
200	0	0	boosted
204	1	0	Demoted again

I/O-Bound Process Timeline			
Time (ticks)	Queue	Ticks Used	Action
0	0	0	Process starts
2	0	2	I/O block (sleep)
4	0	0	Resume (ticks reset, same queue)
6	0	2	I/O block again
8	0	0	Resume (still queue 0)
12	0	0	Completes

Mixed Workload				
Process Type	Final Queue	Completion Time	CPU Time	CPU Time
I/O-bound	0-1	Fast (~50 ticks)	Low	Low
CPU-bound	3	Slow (~500 ticks)	High	High

Part 9: Syscall output screenshot

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ testproc

=== MLFQ Scheduler - Process Info Test ===

Test 1: Initial Process State
=====
Process Information
=====
PID:          3
Name:         testproc
Queue Level:  0 (Highest Priority)
Ticks Used:   0
Time Quantum: 4 ticks
State:        4 (RUNNABLE)
=====

Test 2: After Some CPU Work
=====
Process Information
=====
PID:          3
Name:         testproc
Queue Level:  0 (Highest Priority)
Ticks Used:   0
Time Quantum: 4 ticks
State:        4 (RUNNABLE)
=====

[?] SUCCESS: getprocinfo system call working correctly!
```